

# Merge as Graph Extension

Sasson Margalio

June 2026

## Abstract

This article proposes a formal interpretation of Merge as graph extension inside the Workspace. The Workspace is represented as a graph: elementary syntactic objects are nodes, and complex objects formed by Merge are accessible paths or subgraphs between boundary nodes. When Merge applies, it preserves the previous Workspace and adds a new accessible path that can participate in later applications of Merge.

The same processing cycle is proposed for five internal Layers of the Language Faculty: Semantic Component ( $\Sigma$ ), A-Syntax, A-bar Syntax, Distributed Morphology (DM), and Phonological Component ( $\Phi$ ). Each Layer applies an ordered grammar of Extension Rules, relinearizes the result, and transfers a new graph to the next Layer. The Layers are traversed in one order in generation and in the reverse order in comprehension, while the basic Expand algorithm remains the same.

## 1. Introduction

In standard linguistic notation, a rule such as

(1)  $VP \Rightarrow V NP$

is naturally understood as a parsing rule: when a  $V NP$  configuration is found, it may be recognized as a  $VP$ . The proposal developed here gives this kind of rule a graph-theoretic interpretation. An Extension Rule written as  $Left \Rightarrow Right$  is applied by finding the Right side

in the Workspace and adding the Left side as a new accessible path between the same boundary nodes.

This interpretation makes it possible to treat Merge as an operation of graph extension. Merge does not erase or replace the material already present in the Workspace. It adds structure to the Workspace in a way that preserves earlier paths and makes the newly added path available for later applications of Merge.

Each Layer of the Language Faculty is modelled as an ordered grammar of Extension Rules. A Layer receives a Workspace, applies Merge repeatedly inside that Workspace, and then produces a new graph through Relinearization. Transfer passes that graph as the initial Workspace of the next Layer. The grammars differ from Layer to Layer, but the basic processing cycle remains the same.

The article is organized as follows. Section 2 presents the five-Layer architecture of the Language Faculty. The Workspace is introduced in Section 3. Section 4 distinguishes Layers, grammars, Expand, and the initial Workspace. The directed-graph notation appears in Section 5. Sections 6 and 7 give two examples of Merge as the addition of a new path. Layer grammars and the Expand procedure are described in Section 8. Section 9 states the graph-extension interpretation of Merge. Relinearization and Transfer are discussed in Section 10. Section 11 concludes.

## 2. The Five-Layer Architecture

The proposal assumes five Layers internal to the Language Faculty:

- Semantic Component ( $\Sigma$ ),
- A-Syntax,
- A-bar Syntax,
- Distributed Morphology (DM),
- Phonological Component ( $\Phi$ ).

These Layers are distinct from the external systems. **C-I** is the intelligent Agent itself, interacting with other cognitive systems. **A-P** is the biological articulatory-perceptual performance system. The five Layers constitute the internal linguistic machinery between them.

The full generation pipeline is:

(2)

**C-I** → [Agentic Form] →

**Semantic Component ( $\Sigma$ )** → [Event Structure] →

**A-Syntax** → [NP Structure] →

**A-bar Syntax** → [Surface Structure] →

**Distributed Morphology (DM)** → [Phonological Representation] →

**Phonological Component ( $\Phi$ )** → [Phonetic Form] →

**A-P**

The bracketed terms name representations transferred between adjacent components.

In generation, the components are traversed in this order:

(3) **C-I →  $\Sigma$  → A-Syntax → A-bar Syntax → DM →  $\Phi$  → A-P**

C-I supplies Agentic Form to the first internal Layer, the Semantic Component  $\Sigma$ . From that point on, each Layer receives the representation produced by the preceding component, applies its own grammar, and produces the representation required by the next component. At the end of the pipeline, the Phonological Component supplies Phonetic Form to A-P.

In comprehension, the same components are traversed in the reverse order:

(4) **A-P →  $\Phi$  → DM → A-bar Syntax → A-Syntax →  $\Sigma$  → C-I**

In this direction, the derivation begins with Phonetic Form supplied by A-P and moves inward toward C-I. Comprehension therefore does not require a second architecture. It uses the same five internal Layers and the same general processing cycle but traverses the architecture in the opposite direction.

### 3. The Workspace as a graph

Chomsky (2019) uses the term Workspace for the collection of syntactic objects available during a derivation. The present proposal is related to that use of the term but gives the Workspace a specific graph-theoretic interpretation.

In this formalization, syntactic objects are represented as paths or subgraphs inside the Workspace graph. Merge applies to objects available in the Workspace and adds a new accessible path to the same graph. The result is not a replacement of the previous Workspace, but an extension of it.

This interpretation preserves the familiar idea that Merge applies repeatedly and that each new object can participate in later computation. The difference is in the representation of the Workspace. Instead of reducing the derivation to a single current tree or object, the Workspace graph may contain several paths between the same boundary nodes.

Earlier paths therefore remain available in the graph, while newly added paths can themselves become input to later applications of Merge. The Workspace contains the accumulated result of repeated applications of Merge.

#### **4. Layers, Expand, and the initial Workspace**

A Layer is a persistent component of the architecture. The Workspace is the changing graph on which computation takes place.

In the present proposal, a grammar is an ordered set of Extension Rules. Each Layer is associated with two grammars: one for generation and one for comprehension. The five internal Layers —  $\Sigma$ , A-Syntax, A-bar Syntax, DM, and  $\Phi$  — share the same general form of computation, but not the same rules.

The procedure that applies the grammar of a Layer is called Expand. Expand takes a Workspace as input and applies the ordered grammar of the Layer repeatedly, one application of Merge at a time. When no further rule application is possible, Relinearization constructs a new Workspace, which Expand returns.

The same Expand procedure is used in all five internal Layers. What differs is the grammar supplied by the Layer and the direction of traversal.

For each Layer, the Workspace has an initial state: the state before the first application of Merge in that Layer. This initial Workspace is the representation transferred from the previous Layer, together with the resources expected by the grammar of the present Layer, including the Label nodes used by its Extension Rules.

#### **5. Nodes, Edges, Labels, and Paths**

The Workspace is represented here as a directed graph, consisting of nodes and directed edges. A node is written in the form:

(5) (classname:id)

The unique key of a node is the pair consisting of its classname and its id. The id alone does not identify the node. For example, (token:1) and (label:1) are different nodes, because their classnames are different. Within a single graph, there is at most one node with the same classname-id pair.

In this article, only three classnames are used: token, label, and control. Token nodes represent positions in a path. Label nodes represent labels by which tokens are recognized by rules. Control nodes mark the beginning and end of the Workspace graph.

Examples of well-formed nodes are:

(6)  
(token:1)  
(token:2)  
(label:E)  
(label:F)  
(control:begin)  
(control:end)

In the examples below, token ids are written as running integers. A directed edge is indicated by an arrow:

(7) (token:1) → (token:2)

A path is a sequence of nodes connected by directed edges. For example:

(8) (control:begin) → (token:1) → (token:2) → (control:end)

represents a path from (control:begin) to (control:end) through the token nodes (token:1) and (token:2).

Labelling is also represented by directed edges. Thus a token associated with the label E is represented by an edge from the token node to the label node:

(9) (token:1) → (label:E)

A graph containing both the path and the labels can therefore be written explicitly as the following set of edges:

(10)  
(control:begin) → (token:1)  
(token:1) → (token:2)  
(token:2) → (control:end)

(token:1) → (label:E)

(token:2) → (label:F)

For readability, I also use a compact curly-brace notation. An additional outgoing edge may be written immediately after the source token node. In this article, the curly-brace notation is normally used for an edge from a token node to a label node:

(11) (token:1) {label:E}

This is an abbreviation for:

(12) (token:1) → (label:E)

Using this abbreviation, the graph in (11) can be written more compactly as:

(13) (control:begin) → (token:1) {label:E} → (token:2) {label:F} → (control:end)

The arrow in (13) gives the main directed path. The curly-brace expression displays an additional outgoing edge from the preceding token node to the label node named inside the braces.

## 6. Extension Rules and a Simple Merge Example

Rules retain familiar linguistic notation:

(14) Left => Right

The interpretation is:

(15) find Right, add Left

Thus, a rule such as:

(16) A => B C

is applied by finding a path labelled B C in the Workspace and adding a path labelled A at the same location. The rule does not replace B C with A. It extends the Workspace so that A becomes available as an additional path. In this sense, Merge is interpreted as graph extension.

Assume the initial Workspace contains the following graph:

(17) (control) → (token:1) {label} → (token:2) {label} → (control)

This graph contains a path from (control) to (control) through two token nodes, labelled B and C. The right side of the rule in (16) is therefore found in the Workspace:

(18) (token:1) {label} → (token:2) {label}

Merge then adds a fresh token node labelled A, creating an additional path between the same boundary nodes:

(19) (control) → (token:3) {label} → (control)

After Merge, the Workspace contains both paths:

(20)

(control) → (token:1) {label} → (token:2) {label} → (control)

(control) → (token:3) {label} → (control)

The original path has not been erased. It remains in the Workspace, while the newly added path makes A available for later applications of Merge.

In this example, the boundary nodes are (control) and (control). More generally, Merge can apply between any two nodes X and Y, provided there is a path from X to Y matching the right side of an Extension Rule. Merge finds such a path and adds another path between those same boundary nodes.

The same interpretation applies in both directions of traversal. Generation and comprehension use different ordered sets of Extension Rules, but the basic form of Merge as graph extension remains the same.

## 7. Merge Inside a Larger Workspace

The previous example used (control:begin) and (control:end) as the boundary nodes. That is only the simplest case. In general, Merge can apply inside a larger Workspace.

Consider a simplified but more grammar-like example. Suppose the Workspace contains the following path:

(21)

(control:begin) → (token:1) {label:pronoun} → (token:2) {label:verb} → (token:3) {label:Det} →

(token:4) {label:noun} → (control:end)

Suppose also that the Layer contains the Extension Rule:

(22) DP => Det noun

For this application of Merge, the relevant boundary nodes are:

(23)

X = (token:2)

Y = (control:end)

The boundary node X is the node immediately before the matching span, and Y is the node immediately after it. Between these two boundary nodes, the Workspace contains the matching span:

(24) (token:3) {label:Det} → (token:4) {label:noun}

Merge then adds a fresh token node labelled DP, creating an additional path between the same boundary nodes:

(25) (token:2) → (token:5) {label:DP} → (control:end)

After this application of Merge, the Workspace contains both the original path segment:

(26) (token:2) → (token:3) {label:Det} → (token:4) {label:noun} → (control:end)

and the newly added path segment:

(27) (token:2) → (token:5) {label:DP} → (control:end)

The Workspace is therefore not limited to the material directly involved in the current application of Merge. It contains all accumulated paths in the graph. A particular application of Merge is localized by two boundary nodes: it finds the right side of a rule between those boundaries and adds the left side as a new path between the same boundaries.

## 8. Layer Grammars and Expand

Section 4 introduced the distinction between a Layer, its grammars, and the Workspace on which computation takes place. This section states the corresponding procedure more explicitly.

Each Layer has two ordered grammars of Extension Rules: one for generation and one for comprehension. The generation grammar is used when the architecture is traversed from



C-I toward A-P. The comprehension grammar is used when the architecture is traversed from A-P toward C-I. The comprehension grammar is therefore parsing-like, but it belongs to the same Layer and uses the same general form of Extension Rules.

For a given Layer and direction of traversal, Expand proceeds as follows:

- receive a Workspace;
- select the grammar of the present Layer for the current direction of traversal;
- scan the selected ordered grammar and apply Merge repeatedly inside the Workspace;
- when no further rule application is possible, apply Relinearization;
- return the resulting graph as the Output Graph.

The Workspace received by a Layer contains both the material transferred from the previous Layer and the Label nodes expected by the grammar of the current Layer. A particular application of Merge may introduce a fresh token node and link it to an existing Label node, but the Label node itself is already present.

What differs from Layer to Layer is the ordered grammar supplied to Expand. What remains the same is the processing procedure. In this sense, the five internal Layers share the same general computational form while differing in their rule inventories.

## 9. The Graph-Extension Interpretation of Merge

In the Workspace conception of Merge, objects already available in the Workspace are used to make a new object available. In the graph interpretation, this means that a path already available in the Workspace is used to add another path between the same boundary nodes.

Consider a simple lexical path:

(28) (control:begin) → (token:100) {label:the} → (token:101) {label:book} → (control:end)

Here the words “the” and “book” are not token ids. They are labels. Written without the curly-brace abbreviation, the relevant label edges are:

(29)

(token:100) → (label:the)

(token:101) → (label:book)

Now suppose the Layer contains the following Extension Rules:

(30)

Det => the

noun => book

DP => Det noun

The first rule finds a token labelled the and adds a new token labelled Det over the same span:

(31) (control:begin) → (token:102) {label:Det} → (token:101)

The original token (token:100) is not removed. The Workspace now contains both the original lexical path through (token:100) and the new path through (token:102).

The second rule finds a token labelled book and adds a new token labelled noun over the same span:

(32) (token:102) → (token:103) {label:noun} → (control:end)

At this point, the Workspace contains a path whose relevant label sequence is Det noun:

(33) (control:begin) → (token:102) {label:Det} → (token:103) {label:noun} → (control:end)

The third rule can now apply. It finds the path labelled Det noun and adds a fresh token node labelled DP between the same boundary nodes:

(34) (control:begin) → (token:104) {label:DP} → (control:end)

After these applications, the Workspace contains several paths, including the original lexical path:

(35) (control:begin) → (token:100) {label: the} → (token:101) {label: book} → (control:end)

the category path:

(36) (control:begin) → (token:102) {label:Det} → (token:103) {label:noun} → (control:end)

and the DP path:

(37) (control:begin) → (token:104) {label:DP} → (control:end)

The important point is that Merge does not replace one path with another. The lexical path remains in the Workspace, the category path is added, and the DP path is then added as another accessible path between the same boundary nodes. Since each newly added path may make further Extension Rules applicable, Expand continues as long as further applications of the Layer's grammar remain possible.

## 10. Relinearization and Transfer

Relinearization is the construction of a new Output Graph from selected paths in the extended Workspace.

After Merge operations have extended the Workspace, several paths may be available between the same boundary nodes. Some of these paths may be interpretable by the next Layer; others may not. Relinearization determines which paths and labels are passed on.

Relinearization walks complete paths from (control:begin) to (control:end), extracts the output labels interpretable by the next Layer, and builds a new graph from them.

The Output Graph is a new Workspace. It is not the same Workspace with some paths marked as visible or invisible. It may contain equivalently named nodes, including equivalently named label nodes and control nodes, but it is a new graph object constructed by Relinearization.

Transfer is the passing of the Output Graph of one Layer as the initial Workspace of the next Layer. When the next Layer receives this Workspace, the transferred material supplies its initial state. The Label nodes expected by the grammar of the next Layer are also present before Merge begins.

Thus, Expand extends the Workspace by repeated applications of Merge; Relinearization constructs a new Output Graph from the extended Workspace; and Transfer passes that Output Graph to the next Layer.

## 11. Conclusion

This article has proposed a formal interpretation of Merge as graph extension inside the Workspace. The Workspace is represented as a directed graph, and Merge adds a new accessible path between boundary nodes without erasing the paths already present.

The same general processing cycle was proposed for five internal Layers of the Language Faculty: Semantic Component ( $\Sigma$ ), A-Syntax, A-bar Syntax, Distributed Morphology (DM), and Phonological Component ( $\Phi$ ). These Layers are traversed in one order in generation and in the reverse order in comprehension. The grammars differ from Layer to Layer and by direction of traversal, but the processing cycle remains the same.

In both directions, a Layer receives a Workspace, applies the relevant ordered grammar of Extension Rules, constructs a new Output Graph by Relinearization, and transfers that graph to the next Layer. Merge is therefore not treated as a tree-replacement operation, but as a path-adding operation inside an accumulating graph Workspace.

## **References**

Chomsky, Noam. 2019. The UCLA lectures (April 29–May 2, 2019). Transcribed and edited by Robert Freidin. LingBuzz/005485.

Halle, Morris, and Alec Marantz. 1993. Distributed Morphology and the pieces of inflection. In *The view from Building 20: Essays in honor of Sylvain Bromberger*, ed. Kenneth Hale and Samuel Jay Keyser, 111–176. Cambridge, MA: MIT Press.

Margaliot, Sasson. 2017. Minimalist semantics and pre-linguistic intelligence. LingBuzz/003409.